МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
“Санкт-Петербургский национальный
исследовательский университет информационных
технологий механики и оптики”
Мегафакультет: трансляционных информационных
технологий
Факультет: информационных технологий и
программирования

Университет ИТМО

По дисциплине: “Инструментальные средства разработки ПО”
Тема "Документирование"

Выполнил студент группы №M3121:
Галочкин Тимофей Антонович

Работу проверила:
Алексеева Милена Константиновна

САНКТ-ПЕТЕРБУРГ
2026

1 Цель работы

- Научится писать, как отдельную документацию, так и внедрять её в исходный код

2 Документация к файлу mysterious_file.py

2.1 Функция bought_products

Функция вычитает сумму покупки из денег пользователя.

```
    def bought_products(money, price_per_item, items_count):
        total = price_per_item * items_count
        if total > money:
            return "Недостаточно денег"
        return money - total
```

Параметры

1. `money (int)` - число денег у пользователя
2. `price_per_item (float)` - цена одного товара
3. `items_count` (int) - число одинаковых товаров

Возвращает

1. `int` или `float` - оставшиеся деньги у пользователя
2. `«Недостаточно денег» (str)` - если денег не хватает

Примеры вызова:

```
        >>> bought_products(100, 10, 5)
        50
```

```
        >>> bought_products(100, 100, 5)
        Недостаточно денег
```

2.2 Функция is_in_list

Делает проверку на наличие элемента в заранее заготовленном списке, состоящим из дней недель.

```
    def is_in_list(day):
        return day.lower() in ["сб", "вс", "суббота", "воскресенье"]
```

Параметры

1. `day (str)` - Элемент, наличие которого мы хотим проверить (День недели)
2. `arr = ["сб "вс "суббота "воскресенье"] (list):` - Массив дней недели

Возвращает

1. `True (bool)` - Элемент есть в массиве
2. `False (bool)` - Элемента в массиве нет

Примеры вызова:

```
>>> is_in_list("Суббота")
True
```

```
>>> is_in_list("С")
False
```

2.3 Функция is_even

Функция делает проверку на то, чётное ли число, и выводит результат

```
def is_even(parametr):
    return parametr % 2 == 0
```

Параметры

- `parametr (int)` - Элемент, который мы хотим проверить

Возвращает

1. `True (bool)` - Число кратно 2
2. `False (bool)` - Число не кратно 2

Примеры вызова:

```
>>> is_even(2)
True
```

```
>>> is_even(2)
False
```

2.4 Функция multiples_of_3

Функция возвращает список чисел, кратных трём

```
def multiples_of_3(prm1, prm2, step = 1):
    res = []
    for i in range(prm1, prm2, step):
        if i % 3 == 0:
            res.append(i)
    return res
```

Параметры

1. `prm1 (int)` - Начало диапозона перебора индексов
2. `prm2 (int)` - Конец диапозона перебора индексов
3. `step (int)` - Шаг перебора индексов
    - Если при вызове ф-ции шаг *(step)* **не обозначаем**: step = 1
    - Если при вызове ф-ции шаг *(step)* **обозначаем**: step = <введённое число>

Возвращает

- `list` - Список чисел, которые делятся на 3

Примеры вызова:

```
>>> multiples_of_3(0, 13)
[0, 3, 6, 9]
```

```
>>> multiples_of_3(0, 13, 2)
[0, 6, 12]
```

```
>>> multiples_of_3(0, 13, 2)
[0, 6, 12]
```

2.5 Функция sentiment_score

Функция возвращает коэффицент (оценку) того, на сколько сильно текст был эмоционально окрашен

```
def sentiment_score(txt, pos, neg):
    words = txt.lower().split()
    ratio = 0

    for word in words:
        if word in pos:
            ratio += 1
        elif word in neg:
            ratio -= 1

        if word.endswith('!'):
            ratio *= 1.5
        elif word.endswith('?'):
            ratio *= 0.3

    return round(ratio, 1)
```

Параметры

1. `txt (str)` - Строка, которую будет оценивать функция
2. `pos (list)` - Список положительных слов (прилагательных)
3. `neg (list)` - Список негативных слов (прилагательных)

Возвращает

- `float` - Коэффицент эмоциональной окраски текста, округлённый до десятых

Примеры вызова:

```
>>> sentiment_score("I love Python!", ["love", "like", "enjoy",
↪  "adore", "appreciate"], ["hate", "dislike", "detest",
↪  "antipathy"])
1.5
```

```
>>> sentiment_score("love like enjoy", ["love", "like", "enjoy"],
↪  ["hate"])
3.0
```

```
>>> sentiment_score("Do I love! Python", ["love"], ["hate"])
0.0
```

2.6 Функция sentiment_score

Функция делает перевод числа из десятичной системы счистления в двоичную и выводит результат. В текущей версии переменная count не выводится

```
def transform_into_binary(num, count = 0):
    res = []

    while num > 0:
        res.append(num % 2)
        num //= 2
        count += 1

    if count > 10:
        res.reverse()

    return ''.join(map(str, res)) if res else "0"
```

Параметры

1. `num (int)` - Число в десятичной системы счистления
2. `count (int)` - Список положительных слов (прилагательных)
3. `neg (list)` - Количество итераций

Возвращает

- `str` - Строка из символов входящего числа в двоичной системе счистления

Примеры вызова:

```
>>>transform_into_binary(10)
0101
```

```
>>> transform_into_binary(12)
('1100', 4)
```

```
>>> transform_into_binary(12)
('1100', 4)
```

```
>>> transform_into_binary(12, 23)
('1100', 27)
```

2.7 Граф истории коммитов

На рисунке показана история коммитов в ветке `doc_561160`.

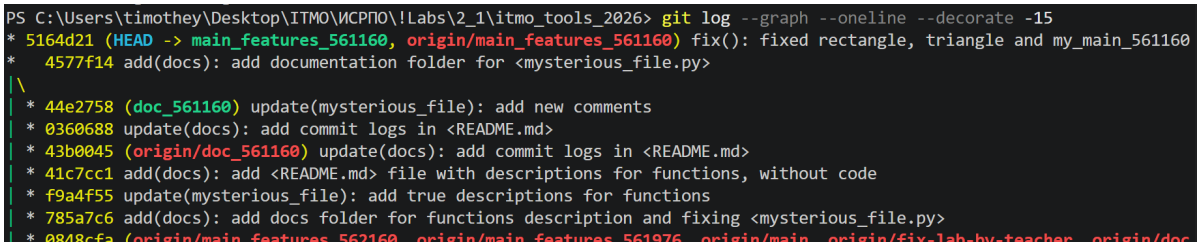


Рис. 1: История коммитов в ветке doc_561160

---

3 Проблемы и трудности

Информация

Поскольку с **LaTeX** и **Markdown** до написания лабораторной работы я не был знаком, поэтому всю информацию нужно было искать самостоятельно и тщательно делать выборку из большого количества источников. Также, я принял решение не только читать статьи на сайтах с описанием команд и документациями, но и обращаться к **LLM Hangzhou DeepSeek Artificial Intelligence Basic Technology Research Co., Ltd**, чтобы облегчить поиск нужных команд или для понимания нюансов работы с **LaTeX** и **Markdown**. Более того, на сайте https://ru.overleaf.com/ редактор предлагает вставить определённые теги, что позволет "поиграться"и попробовать в действии их посмотреть из-за встроенного на сайте компилятора для **LaTeX**, что значительно упрощало поиск подходящих тегов.

---

4 Выводы

При написании данной лабораторной работы я сталкивался со множеством трудностей, начиная с понимания того, что делал имел в виду старый разраб, когда создавал эти функции, заканчивая правильностью использования тех или иных тегов при работе с **LaTeX** и **Markdown**. Благодаря этой работе я познакомился с языками разметки **LaTeX** и **Markdown** и выучил меня базовым теги, которые пригодятся в документации реального проекта.